Notes and Exercises from *Computability, Complexity, and Languages* by Martin D. Davis et al.

Anthony Ozog

Chapter 2: Programs and Computable Functions

Section 1. A Programming Languages

Notes

The first programming language that is defined is pretty simple. It only has the following parts

$V \leftarrow V + 1$	Increment	
$V \leftarrow V - 1$	Decrement*	1
IF $V \neq 0$ GOTO A	Conditional Branch	

* If $V = 0$ then keep $V = 0$.
Actually there are no negative numbers, they only kind of numbers the program works for are just in $\mathbb{N}$. Also for convention they call the input variables $X_1, X_2, X_3, \ldots$ and the output variables $Y_1, Y_2, Y_3, \ldots$ and local variables $Z_1, Z_2, Z_3, \ldots$ often dropping the subscript.

X	Input	
Y	Output	2
Z	Local	

Also instructions are optionally "labelled" using the following notation.

$$A_1, B_1, C_1, D_1, E_1, A_2, B_2, C_2, D_2, E_2, A_3, \ldots \quad 3$$

Once again often with the subscript dropped. E is often the label of the "exit" instruction, meaning it has no instructions and halts the program. Also instructions must be finite. Additionally local and output variables are initialized to 0. Finally this programming language is designated $\mathscr{L}$.

Section 2. Some Examples of Programs

Notes

Abbreviating a "pseudo"-instruction or set of instructions is called at *Macro* and the following is an example of a common one.

$$\text{GOTO } [L] \tag{4}$$

Where the IF part of the conditional branch is omitted. Another common macro that is seen a lot is

$$V \leftarrow V' \tag{5}$$

where the V' is replaced by some other variable during the *macro expansion*. Here the books works hard to show how this assignment/initialization takes place and what the expansion looks like. As a mathematician it seems pretty simple that I can initialize a variable to whatever I want it to be, including outputs of functions on variables denoted $+, -, \times, ...$, but it is cool to see how these abstractions occur in the language $\mathscr{L}$.

Exercises

1. Write a program in $\mathscr{L}$ that computes the function $f(x) = 3x$.
   Solution

$$\begin{array}{ll} & Y \leftarrow X \\ & Z \leftarrow 2 \\ & \left.\begin{array}{l}\text{IF } Y \neq 0 \text{ GOTO } A \\ \text{GOTO } E\end{array}\right] \text{ Unnecessary} \\ [A] & Y \leftarrow X + Y \\ & Z \leftarrow Z - 1 \\ & \text{IF } Z \neq 0 \text{ GOTO } A \end{array} \tag{6}$$

2. Write a program in$\mathscr{L}$ that solves Exercise 1 using no macros.
   I am not gonna do this one since you can just fill in the macros.
3. Let $f(x) = 1$ if x is even; $f(x) = 0$ if x is odd. Write a program in $\mathscr{L}$ that computes f.

$$\begin{array}{ll} [C] & \text{IF } X \neq 0 \text{ GOTO } A \\ & \text{GOTO E} \\ [A] & X \leftarrow X - 1 \\ & Y = Y + 1 \\ & \text{IF } X \neq 0 \text{ GOTO } B \\ & \text{GOTO E} \\ [B] & X \leftarrow X - 1 \\ & Y = Y - 1 \\ & \text{IF } X \neq 0 \text{ GOTO } A \end{array} \tag{7}$$

4. Let $f(x) = 1$ if x is even; $f(x)$ undefined if x is odd. Write a program in $\mathscr{L}$ that computes f.

```
        Y ← 1
[C]     IF X ≠ 0 GOTO A
        GOTO E
[A]     X ← X − 1
        IF X ≠ 0 GOTO B
        GOTO A
[B]     X ← X − 1
        IF X ≠ 0 GOTO A
```

8

Section 3. Syntax

Notes

This section is a bit of repetition as well as a very detailed account of the language $\mathscr{L}$, so I will have brief notes.
The following are called *statements*

$$\begin{aligned} &V \leftarrow V + 1 \\ &V \leftarrow V - 1 \\ &V \leftarrow V \\ &\text{IF } V \neq 0 \text{ GOTO } L \end{aligned} \tag{9}$$

where V may be any variable and L may be any label. An *instruction* is either a statement or a label-statement pair. A *program* is a list (finite sequence) of instructions. The length of the list is the *length* of a program. The *empty program* is the program of length 0.
The *state of a program* $\mathscr{P}$ is a list of equations of the form $V = m$, where V is a variable and m is a number, including an equation for each variable that occurs in $\mathscr{P}$ and including no two equations with the same variable.
Let σ be a state of $\mathscr{P}$ and let V be a variable that occurs in σ. The *value of* V *at* σ is then the number q such that the equation $V = q$ is one of the equations making up σ. The *snapshot* or *instantaneous description* of a program $\mathscr{P}$ of length n is the pair (i, σ) where i is the instruction about to be executed and $1 \leq i \leq n + 1$ and σ is a state of $\mathscr{P}$. $i = n + 1$ is the "stop" instruction.
Page 27 defines all the cases of a *successor* of a snapshot.
A *computation* of a program $\mathscr{P}$ is a list

$$s_1, s_2, s_3, ..., s_k \tag{10}$$

s_i snapshots of $\mathscr{P}$, such that s_{i+1} is the successor of s_i for $i = 1, 2, 3, ..., k - 1$ and s_k is *terminal*, meaning $i = n + 1$.

Exercises

1. Give a program $\mathscr{P}$ that such that for every computation $s_1, s_2, ..., s_k$ of $\mathscr{P}$, $k = 5$.
   Solution

$$\begin{aligned} &Z \leftarrow 0 \\ &Z \leftarrow Z + 1 \\ &Z \leftarrow Z + 1 \\ &Z \leftarrow Z + 1 \end{aligned} \tag{11}$$

Section 4. Computable Functions

The *Initial State* of a program $\mathscr{P}$ in the language $\mathscr{L}$ for $r_1, r_2, ..., r_m$ given numbers is the state σ of $\mathscr{P}$ where

$$X_1 = r_1, X_2 = r_2, ..., X_m = r_m, Y = 0 \tag{12}$$

and $V = 0$ for all other variables in $\mathscr{P}$. The *Initial Snapshot* is denoted $(1, \sigma)$.
Case 1. There exists a computation $s_1, s_2, ..., s_k$ of $\mathscr{P}$ beginning with the initial snapshot. Then $\psi_{\mathscr{P}}^{(m)}(r_1, r_2, ..., r_m)$ is the value of Y at the terminal snapshot s_k.
Case 2. There does not exists a computation. This means there is an infinite sequence $s_1, s_2, ...$ beginning with the initial snapshot. Also, $\psi_{\mathscr{P}}^{(m)}(r_1, r_2, ..., r_m)$ is undefined here.
A function g is *partially computable* if for some $r_1, r_2, ..., r_m$,

$$g(r_1, r_2, ..., r_m) = \psi_{\mathscr{P}}^{(m)}(r_1, r_2, ..., r_m). \tag{13}$$

Also when $g(r_1, r_2, ..., r_m)$ is undefined so is $\psi_{\mathscr{P}}^{(m)}(r_1, r_2, ..., r_m)$, and when g is defined they are equal. A function g is total if it is defined for all input $r_1, r_2, ..., r_m$.

A function is *computable* if it is both partially computable and total

Exercises

1. Let $\mathscr{P}$ be a program

$$\begin{aligned} & & &\text{IF } X \neq 0 \text{ GOTO } A \\ &[A] & &X \leftarrow X + 1 \\ & & &\text{IF X} \neq 0 \text{ GOTO A} \\ &[A] & &Y \leftarrow Y + 1 \end{aligned} \tag{14}$$

   What is $\psi_{\mathscr{P}}^{(1)}$?
   Solution
   It is the nowhere defined function.
2. The same as Exercise 1 for the program

$$\begin{aligned} &[B] & &\text{IF } X \neq 0 \text{ GOTO } A \\ & & &Z \leftarrow Z + 1 \\ & & &\text{IF } Z \neq 0 \text{ GOTO } B \\ &[A] & &X \leftarrow X \end{aligned} \tag{15}$$

   Solution

$$\psi_{\mathscr{P}}^{(1)} = \begin{cases} 0 & \text{undefined} \\ x \neq 0 & 0 \end{cases} \tag{16}$$

3. The same as Exercise 1 for the empty program
   Solution

$$\psi_{\mathscr{P}}^{(1)} = 0 \tag{17}$$

4. Let $\mathscr{P}$ be the program

$$\begin{aligned} & Y \leftarrow X_1 \\ [A] \qquad & \text{IF } X_2 \neq 0 \text{ GOTO } E \\ & Y \leftarrow Y + 1 \\ & Y \leftarrow Y + 1 \\ & X_2 \leftarrow X_2 - 1 \\ & \text{GOTO } A \end{aligned} \tag{18}$$

What is $\psi_{\mathscr{P}}^{(1)}(r_1)$? $\psi_{\mathscr{P}}^{(2)}(r_1, r_2)$? $\psi_{\mathscr{P}}^{(3)}(r_1, r_2, r_3)$?

Solution

$$\begin{aligned} \psi_{\mathscr{P}}^{(1)}(r_1) &= \text{undefined} \\ \psi_{\mathscr{P}}^{(2)}(r_1, r_2) &= \begin{cases} (r_1, 0) & \text{undefined} \\ (r_1, r_2 \neq 0) & r_1 \end{cases} \\ \psi_{\mathscr{P}}^{(3)}(r_1, r_2, r_3) &= \begin{cases} (r_1, 0, r_3) & \text{undefined} \\ (r_1, r_2 \neq 0, r_3) & r_1 \end{cases} \end{aligned} \tag{19}$$

5. Show that for every partially computable function $f(x_1, ..., x_n)$ there is a number $m \geq 0$ such that f is computed by infinitely many programs of length m.

Proof

Let $f(x_1, ..., x_n)$ be some partially computable function such that

$$f(x_1, ..., x_n) = \psi_{\mathscr{P}^*}^{(n)}(x_1, ..., x_n) \tag{20}$$

where $\mathscr{P}^*$ is a program of length l that computes f. Now take $m = l + 1$ and look at the set of all programs S of length m that begin with $\mathscr{P}^*$ and whose final instruction does not change the value of Y (the output) and does not contain a GOTO. It is easy to see that all programs in S compute f since the final instruction does not change Y and $\mathscr{P}^*$ would be terminal at that point. It is also easy to see that S is infinite since the final instruction could be any of the following

$$\begin{aligned} & Z_1 \leftarrow 0 \\ & Z_2 \leftarrow 0 \\ & Z_3 \leftarrow 0 \\ & \vdots \\ & Z_1 \leftarrow 1 \\ & Z_2 \leftarrow 1 \\ & Z_3 \leftarrow 1 \\ & \vdots \\ & Z_1 \leftarrow 2 \\ & \vdots \end{aligned} \tag{21}$$

■

6. (a) For every number $k \geq 0$, let f_k be the constant function $f_k(x) = k$. Show that for every k, f_k is computable.

Proof

Let $k \geq 0$ and consider the function $f_k(x) = k$. It is easy to see that f_k it total since it is equal to k for all x. Now all that is left to show is that f_k is partially computable. Considered the following program $\mathscr{P}$

$$Y \leftarrow k \tag{22}$$

where $\psi^{(1)}_{\mathscr{P}}(x) = k$ thus $\psi^{(1)}_{\mathscr{P}}(x) = f_k$. This means f_k is partially computable and since it is total f_k is computable for any k.

■

Section 5. More about Macros

Observe the following macro.

$$W \leftarrow f(V_1, ..., V_n) \tag{23}$$

where f is a partially computable function. This macro is the abbreviation of the following expansion.

$$\begin{aligned}
& Z_m \leftarrow 0 \\
& Z_{m+1} \leftarrow V_1 \\
& Z_{m+2} \leftarrow V_2 \\
& \vdots \\
& Z_{m+n} \leftarrow V_n \\
& Z_{m+n+1} \leftarrow 0 \\
& Z_{m+n+2} \leftarrow 0 \\
& \vdots \\
& Z_{m+n+k} \leftarrow 0 \\
& \mathcal{Q}_m \\
[E_m] \qquad & W \leftarrow Z_n
\end{aligned} \tag{24}$$

where m is large enough that these variables are never used in the main program.
This next macro makes IF statement look more familiar.

$$\text{IF } P(V_1, ..., V_n) \text{ GOTO } L \tag{25}$$

where $P(V_1, ..., V_n)$ is a computable predicate that evaluates to either

$$\text{TRUE} = 1, \quad \text{FALSE} = 0 \tag{26}$$

which means Equation 25 is the expansion of

$$\begin{aligned}
& Z \leftarrow P(V_1, ..., V_n) \\
& \text{IF } Z \neq 0 \text{ GOTO } L
\end{aligned} \tag{27}$$

Note how all of these *macros* are just abbreviations of the original language $\mathscr{L}$ which means if anything is computable with these macros, or any new ones we define using $\mathscr{L}$, then it is computable in $\mathscr{L}$.

Exercises

1. Let $f(x), g(x)$ be computable functions and let $h(x) = f(g(x))$. Show that h is computable.
   Proof Since f and g are computable there exists

$$\begin{aligned}
f(x) &= \psi^{(1)}_{\mathscr{P}_f}(x) \\
&\text{and} \\
g(x) &= \psi^{(1)}_{\mathscr{P}_g}(x)
\end{aligned} \tag{28}$$

   for programs $\mathscr{P}_f$ and $\mathscr{P}_g$ respectively. Then h can be computed using the following program

$$
\begin{aligned}
&X \leftarrow x \\
&Z \leftarrow \psi^{(1)}_{\mathscr{P}_g}(X) \\
&Y \leftarrow \psi^{(1)}_{\mathscr{P}_f}(Z)
\end{aligned}
\tag{29}
$$

which is easily see to compute h.

2. Show by constructing a program that the predicate $x_1 \leq x_2$ is computable.
   Consider the following program

$$
\begin{aligned}
&& Z_1 &\leftarrow x_1 \\
&& Z_2 &\leftarrow x_2 \\
&& &\text{IF } Z_2 \neq 0 \text{ GOTO } P \\
[P] && &\text{IF } Z_1 \neq 0 \text{ GOTO } S \\
[T] && Y &\leftarrow 1 \\
&& &\text{GOTO } E \\
[F] && Y &\leftarrow 0 \\
&& &\text{GOTO } E \\
[S] && Z_1 &\leftarrow Z_1 - 1 \\
&& Z_2 &\leftarrow Z_2 - 1 \\
&& &\text{IF } Z_1 \neq 0 \text{ GOTO } C \\
&& &\text{GOTO } T \\
[C] && &\text{IF } Z_2 \neq 0 \text{ GOTO } S \\
&& &\text{GOTO } F
\end{aligned}
\tag{30}
$$

Chapter 3: Primitive Recursive Functions

Section 1. Composition